

SOPHGO BM1684X: A Commercial High Performance Terminal AI Processor with Large Model Support

Peng Gao

*State Key Laboratory of Networking and Switching Technology
Beijing University of Posts and Telecommunications,
SOPHGO TECHNOLOGIES PTE. LTD.
Beijing, China
peng.gao@sophgo.com*

Yang Liu

*State Key Laboratory of Networking and Switching Technology
Beijing University of Posts and Telecommunications
Beijing, China
liu.yang@bupt.edu.cn*

Jun Wang

*SOPHGO TECHNOLOGIES PTE. LTD. SOPHGO TECHNOLOGIES PTE. LTD. SOPHGO TECHNOLOGIES PTE. LTD.
Beijing, China
james.wang@sophgo.com*

Wanlin Cai

*Beijing, China
wanlin.cai@sophgo.com*

Guangchong Shen

*Beijing, China
guangchong.shen@sophgo.com*

Zonghui Hong

*SOPHGO TECHNOLOGIES PTE. LTD. SOPHGO TECHNOLOGIES PTE. LTD. State Key Laboratory of Networking and Switching Technology
Beijing, China
zonghui.hong@sophgo.com*

Jiali Qu

*Beijing, China
jiali.qu@sophgo.com*

Ning Wang

*State Key Laboratory of Networking and Switching Technology
Beijing University of Posts and Telecommunications
Beijing, China
wang.ning@bupt.edu.cn*

Abstract—This paper presents BM1684X, a cutting-edge AI processor from SOPHGO designed to meet the demanding requirements of broad AI applications. Firstly, we employ SIMD architecture with very large data width to design our TPU to reduce the area ratio of the instruction unit and greatly improves the computing power density. Secondly, the customization of special acceleration instructions within the EU enables the dynamic pipeline execution, leading to a reduction in the total number of instructions and execution time. This customization enhances the performance of TPU in processing RQ and DQ operations, crucial for AI computations. Thirdly, the CUBE array within the TPU implements the multiplication and addition operations of 64 pairs of INT8 operands in the channel dimension of the feature map. By utilizing an addition tree instead of a conventional adder, the implementation significantly reduces both area and power consumption, optimizing the efficiency of TPU. Additionally, the BM1684X processor incorporates a 64-input, 64-output, 8-bit crossbar within the lane, facilitating high-performance data gathering. This crossbar design enhances data gathering capabilities, enabling efficient data processing and manipulation within the TPU architecture. Furthermore, BM1684X offers three distinct memory access modes, showing the processor's versatility in addressing a wide range of AI processing needs and optimizing DRAM utilization for various tasks and workloads. Finally, we design a TPU-MLIR toolchain, highlighting its rich features such as unified processing of multiple frameworks, hierarchical design of model abstractions, correctness guarantees, and traceability of each transformation step. BM1684X excels in providing high-

performance computing for a variety of AI models including large models, demonstrating its capabilities through comprehensive evaluations with industry-leading peers.

Index Terms—AI processor, TPU, instruction set, microarchitecture, toolchain.

I. INTRODUCTION

The evolution of AI (artificial intelligence) has marked a significant journey through various stages of development. Initially conceptualized as a field of computer science aimed at mimicking human intelligence, AI has progressed from rule-based systems to more sophisticated approaches. The breakthrough moment arrived with the advent of deep learning, where large-scale neural networks, commonly known as big or large models, emerged as a driving force [17], [18].

The development of deep learning has gone through several stages: 1) In 2012, Alexnet entered the first stage with its leap-forward accuracy improvement in the imagenet competition. The more important AI models during this period include CNN, RNN, and LSTM, which have surpassed humans in visual perception tasks and has certain language understanding [42]; 2) With the publication of Attention is all you need [51], Transformer-type networks began to emerge. Transformer-type AI models are as accurate as CNN in the visual field, and their capabilities in the field of natural

Yang Liu is the corresponding author.

language processing have been greatly improved. However, the scale of Transformer networks is not large at this time; 3) The large model stage is marked by ChatGPT and some recently released multi-modal large model networks [54]. They have made leaps in the understanding and generation of images and natural language. Their efficacy has sparked an exploration of cloud and terminal computing opportunities.

The proliferation of AI technologies has paved the way for a surge in AI PC (AI-powered personal computers). Companies across industries have recognized the potential of integrating AI capabilities into personal computers, fostering local processing, and enabling quicker decision-making. This trend has gained momentum, and as of 2023, numerous companies are actively driving initiatives related to AI PC. This wave of AI PC has, in turn, given rise to the concept of edge or terminal AI computing power [55]. The focus on bringing AI capabilities to the edge, or terminal devices, has become a strategic priority. The advantages of local AI processing, such as reduced latency, enhanced efficiency, and improved privacy, have fueled the push towards embedding AI capabilities directly into devices like smartphones, IoT devices, and other terminals. The transformative journey from the emergence of large models to the widespread integration of AI PC and the subsequent emphasis on terminal AI computing power reflects the dynamic evolution of AI technologies. As companies continue to invest in these advancements, the implications for industries, individuals, and society at large are substantial.

BM1684X is designed by SOPHGO to excel in an extensive range of tasks, from image classification [6] and object detection [56] to instance segmentation [25], semantic segmentation [34], behavioral analysis [7], text recognition [43], language processing [41], speech recognition [5], speech synthesis [49], and search recommendation [30]. This broad spectrum of capabilities positions it as a powerful tool for numerous domains, ensuring that it remains at the forefront of technological advancement.

The contributions of our work are as follows:

- By utilizing an SIMD (Single Instruction Multiple Data) architecture with very large data width, we engineer our TPU to effectively reduce the area ratio of the instruction unit while substantially enhancing the computing power density.
- We customize special acceleration instructions in EU (Execution Unit) to execute dynamic pipeline to decrease the total number of instructions and execution time, improve the performance of TPU processing RQ (requantization) and DQ (dequantization). The CUBE array implements the multiplication and addition operation of 64 pairs INT8 operands in the channel dimension of feature map. The implementation of the operation adopts the addition tree instead of the ordinary adder, which greatly reduces the area and power consumption. BM1684X provides a 64-input, 64-output, 8-bit crossbar inside the lane, enabling high performance data gather.

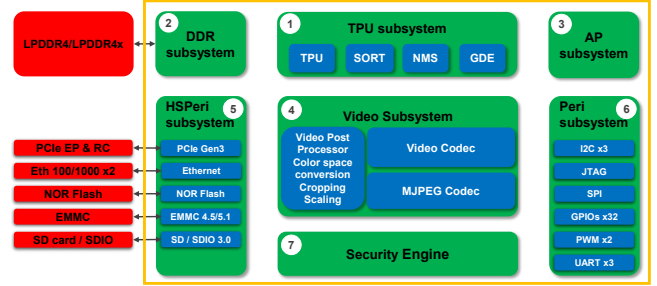


Fig. 1: Functional block of BM1684X.

- BM1684X provides three memory access modes to demonstrate the processor's flexibility in catering to diverse AI processing requirements and optimizing DRAM utilization for different tasks and workloads.
- The TPU-MLIR toolchain introduces a new dimension to BM1684X's capabilities with its multi-framework processing approach. This toolchain supports various neural network frameworks, and its layered design abstracts the network model into tiers, enabling efficient processing.
- We carry out performance evaluations with popular AI models across various batch sizes comprehensively assess the product's efficiency under different workloads including large models with industry-leading peers.

The rest of this paper is organized as follows: Sec. II describes the SoC of our chip. Sec. III presents our TPU subsystem. Sec. IV introduces our toolchain. Sec. VI discusses related work. Sec. VII concludes the paper and discusses lessons learned.

II. SoC

BM1684X integrates a rich array of functional modules, each tailored to address specific needs within the realm of high-performance computing and beyond, as shown in Fig. 1. Modules include DDR subsystem, TPU (Tensor Processing Unit) subsystem, AP (Application Processor) subsystem, HSPeri (High Speed Peripheral) subsystem, Peri (Peripheral) subsystem, video subsystem and security engine. The DDR subsystem provides data transfer between memory and processing units, with a total bandwidth of 68.256 GB/s. The TPU subsystem is capable of performing complex mathematical operations, with vector computing power of 2TFLOPS@FP32, 4TFLOPS@FP16/BF16, and 8TOPS@INT8, and matrix computing power of 32TOPS@INT8 and 16TFLOPS@FP16/BF16. The AP subsystem is composed of ARM's 8-core A53 processor, clocked at 2.3GHz, providing the computing power required for general computing tasks. The HSPeri subsystem enables connectivity to external devices and peripherals, supports PCIe Gen3 X16 endpoint or PCIe Gen3 X8 endpoint plus PCIe Gen3 X8 root complex. The Peri subsystem expands the BM1684X's ability to interface with a variety of devices. The video subsystem is responsible for video processing and can facilitate decoding and encoding of high-definition videos. The

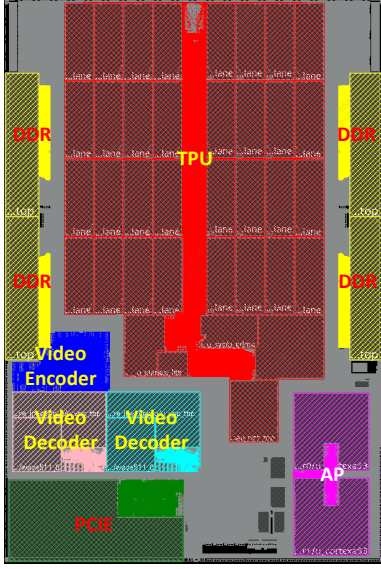


Fig. 2: Die photo of BM1684X.

security engine ensures data protection and secure operations. Fig. 2 shows the die photo of BM1684X which uses a 12nm process and has a die size of 175 mm², 40% of which is used for TPU. Its power consumption is highly correlated with the load of TPU, when the TPU is running at full load, the power consumption is about 25W at the ambient temperature of 25°C.

Our proposed BM1684X pay attention to two typical service scenarios. The former scenario is to decode video and image input in compressed format first, and then perform AI pre-processing on the decoded image. Typical operations are color space conversion, image cutting, and scaling. Then the deep learning AI algorithm is used to process the image to realize functions such as image classification, target recognition, OCR and so on. The latter scenario is to process the data that has been pre-processed or does not need to be pre-processed by using deep learning AI algorithms, which is more suitable for AI algorithms such as natural language processing and transformer, where the performance bottleneck is DRAM bandwidth, especially when edge computing scenario is running with batch size of only one, and does not require a video codec.

For the two scenarios, the on-chip functional modules have different requirements for DRAM access. In the former scenario, video or image decoding module, image pre-processing module, TPU module, AI operator hardware acceleration module are the main modules to access DRAM, and how to achieve reasonable bandwidth allocation and meet the access delay requirements of each module are problems to be solved. In the latter business scenario, only the TPU module and AI operator hardware acceleration module are the main modules for accessing DRAM. BM1684X provides three working modes, each catering to specific memory access patterns and workloads, as shown in Fig. 3. The memory

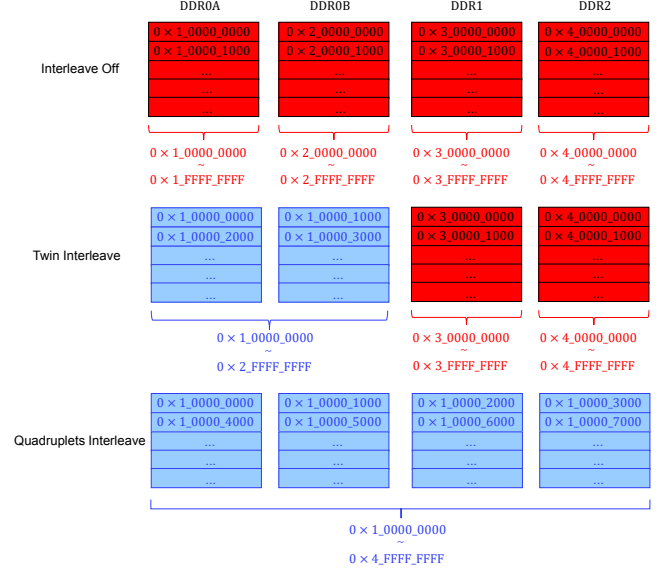


Fig. 3: DDR working modes.

subsystem consists of four separate DDR channels, including DDR0A, DDR0B, DDR1 and DDR2, providing support for high-speed data access and storage. **Interleave Off Mode** is operated with address interleave turned off across all DDR channels. This configuration ensures that each DDR channel is accessed independently, without interleaving addresses. Interleave off Mode is mainly used for debugging, because in this case, it is easier to control which DRAM channel function modules access, and easier to locate bugs. **Twin Interleave Mode** is operated with DDR0A and DDR0B being interleave-addressed in 4KByte chunks, while DDR1 and DDR2 remain uninterleaved. This mode is often employed when the TPU subsystem requires efficient access to feature maps and weight data, which are primarily placed in DDR0A and DDR0B, while DDR1 and DDR2 serve the memory access needs of the TPU subsystem, video subsystem, and HSPeri subsystem. The DDR access bandwidth of the Video subsystem and TPU subsystem are separated from each other and do not compete for each other's bandwidth in this mode. **Quadruplets Interleave Mode** interleaves addresses in 4KByte increments across all four DDR channels. By combining the memory resources of all channels, this mode maximizes memory capacity utilization and minimizes memory access contention among different subsystems, which is helpful for the TPU module and AI operator hardware acceleration module to make full use of all DRAM bandwidth in the latter scenario. The data flows in two modes are shown in Fig. 4.

III. TPU SUBSYSTEM

The TPU subsystem architecture is shown in Fig. 5. TPU is responsible for tensor calculations, with 64 lanes of execution and SIMD processing capabilities. The TIU combines the data from 64 lanes into a 64×8 block, which is then broadcast to each lane using a ring bus for matrix multiplication. The

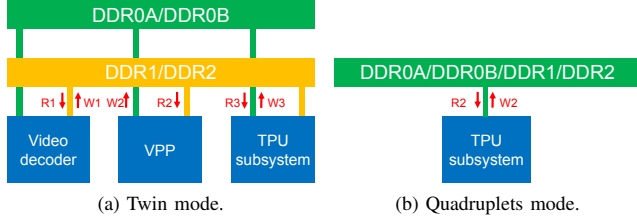


Fig. 4: Data flows in two modes.

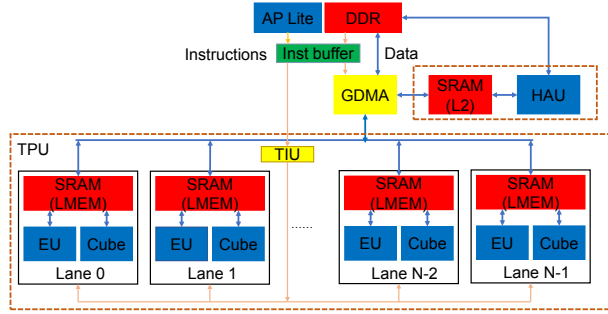


Fig. 5: TPU subsystem architecture.

microarchitecture is shown in the Fig. 6. HAU (Hardware Accelerator Unit) is dedicated to accelerating specific computations critical for AI applications. It includes components such as GDE (Gather Data Engine), SORT (Sorting Engine), and NMS (Non-Maximum Suppression Engine). GDMA (General DMA) is responsible for data movement between different memory hierarchies, including DDR memory, local memory, and L2 cache. AP Lite operates in PIO (Programmed Input/Output) mode, orchestrating the execution of the TPU, GDMA, and HAU modules. It also handles scalar computations as needed. TPU, HAU, GDMA have AXI slave interface for configurations and AP Lite uses AXI bus to control these components.

A. SIMD Architecture with Very Large Data Width

SIMD is a data parallel technique that is widely used in modern computing, especially when dealing with large amounts of data. In the SIMD architecture, a single instruction can operate on multiple data points simultaneously. We design our TPU using SIMD with very large data width. TIU is the control unit, lane is the executive component. The 64 lanes together form a vector parallel computing unit with a width



Fig. 6: TPU microarchitecture.

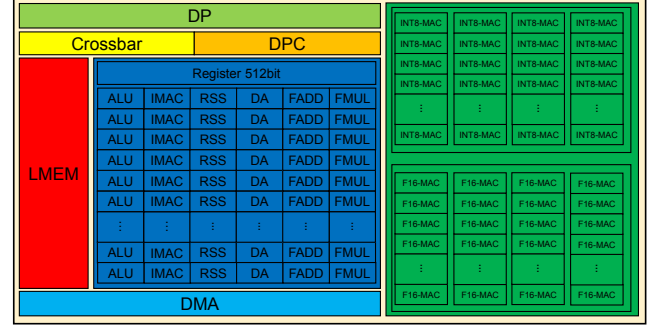


Fig. 7: Microarchitecture of TPU lane.

of 32684 bit. Compared with lane, TIU occupies a small area, and it can be continuously extended to support various updated and complex features, while the need for design modifications inside lane is small.

The rational we employ SIMD with very large data width is that it can process more data in a single operation, significantly increasing data processing throughput. This is especially important for applications that require processing large amounts of data, such as image processing, scientific computing, large-scale data analysis, etc. By processing more data simultaneously, the number of instructions and processing time can be reduced. This parallel processing makes the program run more efficiently, especially when performing vector operations. SIMD with very large data width can perform the same amount of work at lower power consumption than executing multiple independent instructions. This is especially important when energy efficiency is increasingly valued. Using fewer very large width instructions can reduce the complexity of the instruction set and simplify the programming compared to using more small width instructions.

B. Microarchitecture of TPU Lane

Each lane of TPU is composed of DP, DPC, LMEM, DPDMA, EU and CUBE. DP is used to complete the transfer of control information in the lane. When the corresponding lane id meets the requirements of the corresponding bus, the request information is sent to the corresponding processing module. DPC is used to process the operand request of instruction, and realize the reading of data and the writing of results. LMEM is a data storage module for each lane. The size of LMEM in each lane is 256KB and the read/write data bit width is 512 bits. The LMEM has two read ports and two write ports, and cannot access the same bank at the same time. DPDMA is used to process DMA bus requests. EU and CUBE are the main computing arrays, in which EU can support the computation of various operators, and CUBE is specifically used for matrix multiplication and convolution operations. The microarchitecture of TPU lane is shown in Fig. 7.

1) *EU*: Our proposed EU includes ALU, IMAC, RSS, FADD, FMUL, DATA_CONVERT and other computing modules. The operands and *eu_cmd* (the control signal generated by the TIU's decoding of the operator instruction, which *eu_cmd*: TIU收到的指令是面向程序员的 (汇编级), 而 *eu_cmd*是面向硬件电路的Micro-op signals.

	FMUL			FADD			ALU			Result	UCODE0	UCODE1	UCODE2	UCODE3	UCODE4
	OPA	OPB	OPC	OPA	OPB	OPC	OPA	OPB	OPC						
cyc0	NU	NU	ALU(0,0,0-1)	NU	NU	NU	ALU(0,0,0-1)	NU	NU	NU	EU_FMULUC U(0,0,0-1)	EU_ALU(OPX MAGIC_SUB)			
cyc1	NU	NU	NU	NU	NU	NU	NU	NU	NU	NU	EU_FMUL_DLY	EU_FMUL_DLY	EU_FMUL_DLY		
cyc2	RP0X(0,0,0-1)	NU	RP0X(0,0,0-1)	NU	NU	NU	NU	NU	NU	NU	EU_MUL[UC]	EU_MUL[UA]	EU_FMUL A'C)		
cyc3	RP0X(0,0,0-1)	NU	RP0X(0,0,0-1)	NU	NU	NU	NU	NU	NU	NU	EU_END	EU_END	EU_FMUL DLY		
cyc4	RP0X(0,0,0-1)	RP1X(0,0,0-1)	RP0X(0,0,0-1)	NU	NU	NU	NU	NU	NU	NU			EU_FMUL UB A'B)		
cyc5	RP0X(0,0,0-1)	NU	RP0X(0,0,0-1)	NU	NU	NU	NU	NU	NU	NU			EU_FMUL DLY		
cyc6	RP0X(0,0,0-1)	NU	RP0X(0,0,0-1)	NU	RP2X(0,0,0-1)	NU	NU	NU	NU	NU			EU_FADD UB 1.5F-B)		
cyc7	RP0X(0,0,0-1)	NU	RP0X(0,0,0-1)	NU	NU	NU	NU	NU	NU	NU			EU_FADD DLY		
cyc8	RP0X(0,0,0-1)	RP2X(0,0,0-1)	RP0X(0,0,0-1)	NU	NU	NU	NU	NU	NU	NU			EU_FMUL UB A'B)		
cyc9	NU	NU	RP0X(0,0,0-1)	NU	NU	NU	NU	NU	NU	NU			EU_FMUL DLY		
cyc10	RP3X(0,0,0-1)	NU	RP0X(0,0,0-1)	NU	NU	NU	NU	NU	NU	NU			EU_FMUL UA)	EU_FMUL A'C)	
cyc11	RP3X(0,0,0-1)	NU	RP0X(0,0,0-1)	NU	NU	NU	NU	NU	NU	NU			EUN_END	EU_FMUL DLY	
cyc12	RP3X(0,0,0-1)	RP4X(0,0,0-1)	RP0X(0,0,0-1)	NU	NU	NU	NU	NU	NU	NU			EU_FMUL UB A'B)		
cyc13	RP3X(0,0,0-1)	NU	RP0X(0,0,0-1)	NU	NU	NU	NU	NU	NU	NU			EU_FMUL DLY		
cyc14	RP3X(0,0,0-1)	NU	RP0X(0,0,0-1)	NU	RP5X(0,0,0-1)	NU	NU	NU	NU	NU			EU_FADD UB 1.5F-B)		
cyc15	RP3X(0,0,0-1)	NU	RP0X(0,0,0-1)	NU	NU	NU	NU	NU	NU	NU			EU_FADD DLY		
cyc16	RP3X(0,0,0-1)	RP5X(0,0,0-1)	RP0X(0,0,0-1)	NU	NU	NU	NU	NU	NU	NU			EU_FMUL UB A'B)		
cyc17	NU	NU	RP0X(0,0,0-1)	NU	NU	NU	NU	NU	NU	NU			EU_FMUL DLY		
cyc18	RP6X(0,0,0-1)	NU	RP0X(0,0,0-1)	NU	NU	NU	NU	NU	NU	NU			EU_FMUL UA)	EU_FMUL A'C)	
cyc19	RP6X(0,0,0-1)	NU	RP0X(0,0,0-1)	NU	NU	NU	NU	NU	NU	NU			EUN_END	EU_FMUL DLY	
cyc20	RP6X(0,0,0-1)	RP7X(0,0,0-1)	RP0X(0,0,0-1)	NU	NU	NU	NU	NU	NU	NU			EU_FMUL UB A'B)		
cyc21	RP6X(0,0,0-1)	NU	RP0X(0,0,0-1)	NU	RP8X(0,0,0-1)	NU	NU	NU	NU	NU			EU_FMUL DLY		
cyc22	RP6X(0,0,0-1)	NU	RP0X(0,0,0-1)	NU	NU	NU	NU	NU	NU	NU			EU_FADD UB 1.5F-B)		
cyc23	RP6X(0,0,0-1)	NU	RP0X(0,0,0-1)	NU	NU	NU	NU	NU	NU	NU			EU_FADD DLY		
cyc24	RP6X(0,0,0-1)	RP9X(0,0,0-1)	RP0X(0,0,0-1)	NU	NU	NU	NU	NU	NU	NU			EU_FMUL UB A'B)		
cyc25	NU	NU	RP0X(0,0,0-1)	NU	NU	NU	NU	NU	NU	NU			EU_FMUL DLY		
cyc26	NU	NU	RP0X(0,0,0-1)	NU	NU	NU	NU	NU	NU	NU	R(0,0,0-1)			EU_RESULT	

Fig. 8: ucode and data flow of floating-point square root reciprocal.

TIU发出一条指令，所有并行的EU (Lanes) 同时接收 is broadcast by the TIU to EU in all lanes) are input to EU. After decoding the eu_cmd, the operand is passed to the corresponding calculation module, and the subsequent instruction code is transferred along with the result of the calculation unit. When the final result and instruction enter the RES module, it means that all calculations have been completed and can be written back to LMEM. Our proposed EU is shown in Fig. 22 in Appendix A.

The EU uses a data flow architecture, where the data flow is controlled by the ucode which is a micro-instruction from the TIU. With different ucode configurations, fundamental operators in EU can constitute a variety of complex directed acyclic or cyclic atomic operations. Fig. 8 is the ucode and data flow of the floating-point square root reciprocal. Fig. 23 in Appendix B is the ucode and data flow of floating-point division.

Since the data processed by a single instruction in our TPU is a four-dimensional tensor which has a large size, our TPU does not use register file, but directly reads operands from SRAM, and calculates them, and writes back the results to the SRAM. In the vector computation, reducing the number of SRAM reads and writes is very important to improve performance and reduce power consumption. Our TPU reduces SRAM read and write by trying to fuse multiple instructions into one instruction, and by using ucode to control the forward of data between different computation units, such as integral ALU, floating-point ADD, and floating-point MUL.

2) CUBE: CUBE is used for convolution and matrix multiplication computation, supporting INT8, FP16, BF16. The

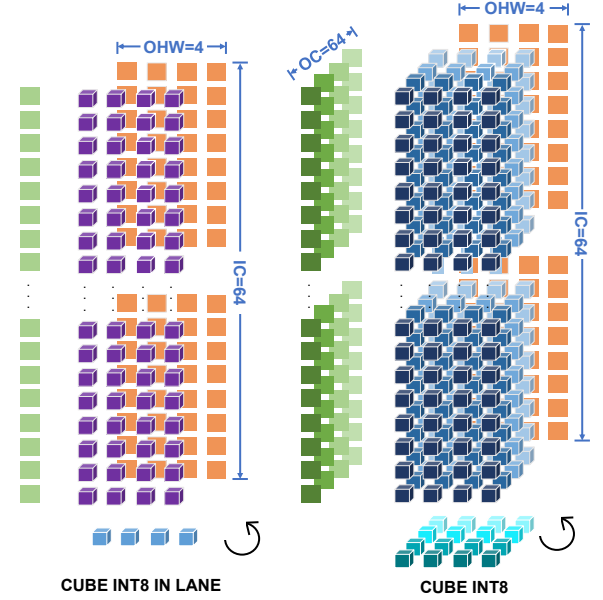


Fig. 9: INT8 convolution/matrix multiplication.

CUBE array for INT8 calculations is 64×4 in size and can compute the partial sum of 4 points simultaneously. FP16 and BF16 share a CUBE array with a size of 32×4 that can compute the partial sum of 4 points simultaneously. When the operand is ready, the operand is sent to the **fixed-point array or floating-point array** for calculation according to the information in cmd, and the number of iterations required is determined by the configuration information and the array is controlled by multiplication.

Each lane implements a 64×4 INT8 MAC (Multiplier Accumulator) array. TPU has 64 lanes, thus there are a total of $64 \times 64 \times 4$ INT8 MAC arrays. At the operating frequency of 1GHz, the computing power is up to 32 TOPS. The CUBE array implements a MAC unit with 64 operands in the channel direction of the feature map. The MAC unit optimizes multiplication and addition in the dimension of 64, which greatly reduces the mac area and power consumption. Weight can be reused for 16 computing cycles, effectively reducing the power consumption of obtaining weight and replacing mac unit operands. The convolution operation can be selected as a simple quantization operation by channel. This quantization operation uses the vector operator and the mac operation pipeline. The quantization operation does not introduce additional cycle, which effectively improves the utilization of CUBE. INT8 convolution/matrix multiplication is shown in Fig. 9.

Similar to the CUBE INT8 array, a 32×4 FP16/BF16 MAC array is implemented on each lane, and a total of $64 \times 32 \times 4$ FP16/BF16 MAC array is implemented with 64 lanes in the TPU, and the computing power is up to 16 TOPS at the operating frequency of 1GHz. The CUBE FP16/BF16 array supports the multiplication and accumulation of 32 dimensions in the direction of the feature map channel. The addition part

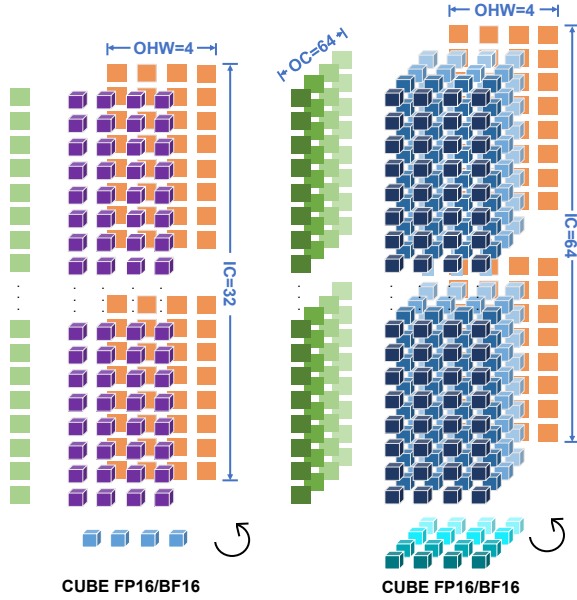


Fig. 10: FP16/BF16 convolution/matrix multiplication.

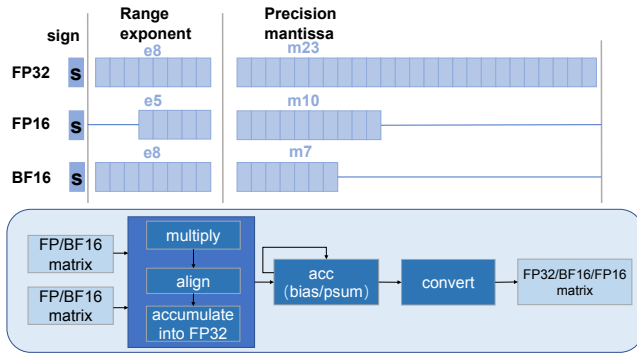


Fig. 11: Multiplicative addition of FP16/BF16 in CUBE and configurable output precision.

is fused, which effectively reduces the area and power consumption of the floating point addition. Weight can be reused up to 16 times, effectively reducing the power consumption of acquiring weight and replacing operand. FP16/BF16 convolution/matrix multiplication is shown in Fig. 10.

In the calculation process of CUBE FP16/BF16, the precision of FP32 is obtained through the realization of floating point addition tree, and the output supports FP16/BF16/FP32, while ensuring the performance, the output precision can meet the needs of different applications. The MAC unit of the CUBE FP16/BF16 adapts to mantissa and exponential widths of FP16 and BF16 data, enabling to support convolution/matrix multiplication of FP16 and BF16 at a small area cost. Multiplicative addition of FP16/BF16 in CUBE and configurable output precision is shown in Fig. 11.

3) **Crossbar**: After the lane data is read from LMEM, it often needs to move the position of the data in the whole vector

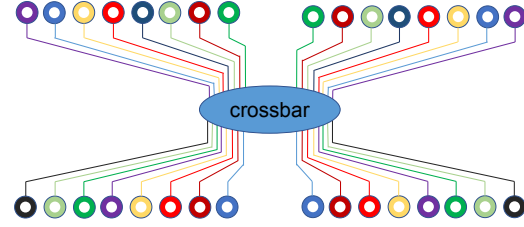


Fig. 12: Crossbar.

before it is provided to EU for calculation. It is often necessary to concatenate the data that is read from LMEM several times after moving, so that the data to be calculated can be obtained. Thus it is very important to design a high performance data aggregation logic to realize high performance pooling and depth-wise convolution operator calculation. We propose a 64-input, 64-output, 8-bit crossbar inside the lane, enabling high performance data aggregation to generate 4 INT8 data. Such performance requires 64 inputs and 64 outputs in each lane. An 8-bit data width crossbar is perfectly acceptable. Our proposed crossbar is shown in Fig. 12.

4) **Dynamic execution pipeline**: To implement RQ and DQ operators, multiple instructions are required. For example, a typical DQ calculation process is to convert an INT32 data to FP32 data, then multiply it by SCALE which is an FP32 data, the result plus ZERO_POINT which is an FP32 data, the result of the FP32 data is converted to INT8 data. Based on the data flow described in the previous section, EU can implement dynamic data flow. By configuring different eu_cmd, we implement different execution pipelines to accelerate different computations. RQ and DQ operators in the EU customize special acceleration instructions for dynamic flow. After acceleration, we can get a calculation result in each cycle.

In our proposed dynamic execution pipeline, data forwarding reduces instruction execution latency by solving the data hazard problem, while instruction fusion reduces the total number of instructions and execution time by merging related instructions. Thus, the performance of TPU processing RQ and DQ is greatly improved, and the instruction density is greatly increased. Figs. 13-16 show the dynamic execution pipeline of RQ and DQ.

With dynamic pipeline technology, we get the following benefits: (1) The logic of single instruction issue is much simplified than multi-instruction issue. (2) For the calculation of DQ and RQ, the performance of a single fusion instruction is 40% higher than that of multiple single instructions without fusion, according to our test. (3) The intermediate data which is stored in an intermediate register but not visible to programmers avoids writing back to SRAM and reduces the power consumption of SRAM read and write by 2/3, according to our test.

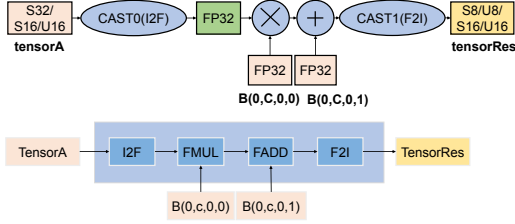


Fig. 13: Dynamic execution pipeline of RQ-0.

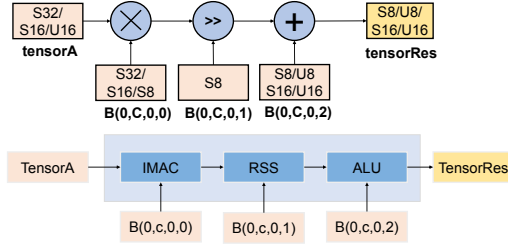


Fig. 14: Dynamic execution pipeline of RQ-1.

C. HAU

We propose SORT, NMS and GDE in our proposed HAU. Since sorting only use the computation of vector, and do not use the computation of matrix multiplication or convolution operation, which TPU is good at. Therefore, we designs a separate arithmetic accelerator in HAU. The supported data types are INT32, UINT32, and FP32. SORT supports ascending and descending order, sorting with index, and TOPK function, and data can be stored in global memory or L2 memory. GDE collects qualified data from the source data based on the values of index and outputs it to the target data. The NMS module in our HAU implements the second half of NMS algorithm, the first half can be completed by our TPU and is used as input to the module. It supports two operations: hard-NMS and soft-NMS. The input of hard-NMS is a mask table of $N \times N$ where N is the number of proposals, each item in the table is INT8, and 1 in the table represents IoU (Intersection over Union) less than threshold or itself. The hard-NMS mask table is shown in Table I. The input of soft-NMS is an $N \times N$ attenuation coefficient table where N is the number of proposals, each item in the table is the attenuation coefficient between the two proposals. Considering the demand for precision and the size of the amount of data transferred, the attenuation coefficient uses FP32. After the attenuation coefficients of all proposals are stored in memory, NMS determines the attenuation values to be used based on the maximum score. If multiple maxima are encountered, the proposal with the lowest address is used as the maximum. After updating the score with the decay value, the NMS continues to look for the maximum value and updates the score until the end condition is met. The soft-NMS mask table is shown in Table II.

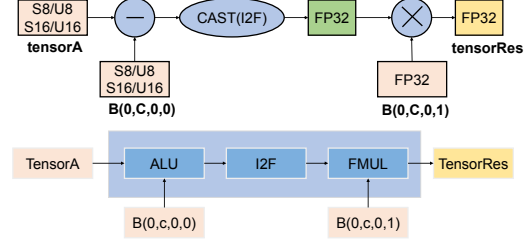


Fig. 15: Dynamic execution pipeline of DQ-0.

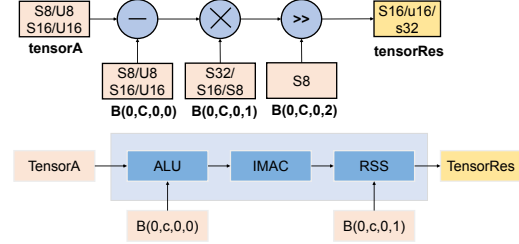


Fig. 16: Dynamic execution pipeline of DQ-1.

D. GDMA

GDMA is an important module to realize the data transfer between DRAM and TPU, and can also realize the data transfer from DRAM to DRAM and the data transfer from TPU to TPU. The data calculation process of a typical operator is to start GDMA to move the data to be calculated from the DRAM to the memory in the TPU. Then the TPU is started for calculation, and the calculation result is saved in the memory inside the TPU. GDMA is then started again to write the result of the calculation back to DRAM. The function of traditional DMA is relatively simple, that is, to realize simple block data transfer, which cannot meet the function and performance requirements of artificial intelligence algorithm accelerator.

Our GDMA has designed unique features for the needs of AI algorithms. Our GDMA supports multidimensional matrix DMA transfer. The data processed by AI algorithms is a multidimensional matrix, i.e. tensor, which is more typical of 2, 3, and 4-dimensional matrices. Due to the algorithm itself, hardware limitations, algorithm optimization and other reasons, the transmitted data is often required to be a multidimensional slice of a multidimensional matrix, and the relevant data is not always stored continuously in DRAM.

Traditional DMA generally only processes continuous data blocks at one time. If traditional DMA is used, it needs to be started by the controller for many times. This control cost is very large and has a serious impact on performance. Another method is to use the linked lists and queue task descriptors, the controller starts DMA once, DMA by reading the linked lists and queue task descriptors, will complete all the tasks one by one, the cost of this way is slightly better than the first in some cases, but in some cases, a large number of linked lists and queue task descriptors are required. The space it occupies may be larger than the data itself, and the overhead of preparing

TABLE I: hard-NMS mask table.

	P_0	P_1	P_2	P_3	P_4	...
P_0	1/addr0	0/addr1	0/addr2	1	1	...
P_1	x/addrN	1/addr(N+1)	0	0	0	...
P_2	x	x	1	0	0	...
P_3	x	x	x	1	0	...
P_4	x	x	x	x	1	...
...	...	...	...	...	...	...

TABLE II: soft-NMS mask table.

	P_0	P_1	P_2	P_3	P_4	...
P_0	1/addr	0.12/addr	0.22	0.33	0.55	...
P_1	...	1/addr	0.11	0.23	0.33	...
P_2	...	...	1	0.11	0.33	...
P_3	...	...	...	1	0.66	...
P_4	...	...	...	...	1	...
...	...	...	...	...	...	...

these descriptors for the controller is very large, and the waste of DRAM access bandwidth is also very large. Therefore, it is necessary to design corresponding data transfer operators based on the requirements of AI algorithms.

In our proposed GDMA, an operator DMA_tensor for data transfer of multidimensional matrix is designed, which can support 4D matrix at most, marking each dimension as N, C, H and W dimensions. Supports functions such as conventional transfer, NC dimension transpose transfer, constant filling, and broadcast transfer of a certain dimension. Compared with traditional DMA, the performance of multi-dimensional matrix data transfer using this operator is greatly improved.

To support AI algorithms efficiently, we design GDMA to replace traditional DMA for the following reasons: (1) In order to pursue computational efficiency, all the data of our TPU are defined as 4-dimensional tensors, and GDMA which abstracts data in TPU or DRAM into 4-dimensional tensors can greatly improve flexibility and reduce the number of DMA instructions by more than an order of magnitude, and thus improves efficiency. For example, transferring a tensor with a channel-dimension of 64 between DRAM and TPU requires at least 64 instructions by using traditional DMA, our GDMA requires only one instruction. Another example is that in AI algorithms of CNN, the data is a 4-dimensional tensor, and we tend to store the data closely contiguously in DRAM. However, due to the limited on-chip SRAM capacity, compilers aiming for the best global performance often slice a tensor calculation, such as a layer of convolution, into multiple subparts for calculation, so the data that we actually need to move from the DRAM to the TPU at one time is a slice of the entire tensor. Our GDMA designed to support 4-dimensional tensor data transfer can be used to implement the data transfer of a 4-dimensional tensor slice by using one instruction, which is completely impossible for traditional DMA. (2) Operators such as Non Zero, Mask Select and Vector Scatter/Gather are often used in the pre- and post-processing of AI algorithms. These operators are not typical SIMD operators, so TPU has many functional limitations when executed. Implementing these operators in GDMA has three advantages: (1) Operators are executed at the same time of data transfer, thus the amount of data transfer

is reduced. (2) The hardware implementation of our GDMA has higher performance than CPU implementation, avoids the performance overhead of operations such as flushing cache, which is required for heterogeneous processors and CPUs to share data. (3) GDMA implementation is not limited to TPU programming model, can support more functions.

We design GDMA to support TPU-TPU direct data transfer for two reasons: (1) Although both GDMA and TPU support data operations such as Mask Sel and Non Zero, their functions and performance are different: TPU can only implement such operations in the SRAM of a lane, while GDMA can perform such operations across lanes. The result of GDMA may need to be stored in the TPU for subsequent operators to calculate. Although such cases are rare, it is valuable in retaining GDMA's ability to support TPU-TPU data transfer. (2) GDMA supports TPU-TPU data transfer, which is one of the data sharing methods of multi-core collaboration. In BM1684X, only a single-core TPU is integrated, so currently it cannot achieve multi-core collaboration to complete the same task. However, in the IP design of TPU and GDMA, multiple TPU cores are supported to work together in tensor parallel and pipeline parallel, for which a task can be split into multiple subtasks by the compiler and run on multiple TPU cores, which are synchronized by handshake protocols and shared by DRAM. We will integrate multi-core TPUs in our next-generation product.

For our next-generation product which integrates multi-core TPUs, we use different working modes according to different application scenarios at the edge as follows, taking two TPU cores as an example: (1) Two TPUs run different models, such as one for object detection and the other for feature extraction. (2) When batch size is 1, the two TPUs run a model in parallel to ensure the full utilization of computing power and on-chip SRAM. (3) When batch size is large, the two TPUs run a model in tensor parallel, our GDMA supports broadcasting the weight data from DRAM to the two TPUs, thereby reducing DRAM access and fully utilizing the computing power and on-chip SRAM. (4) When batch size is 1, if the computing power of one TPU cannot meet the latency requirement, multiple TPUs will be combined for calculation by splitting the one layer of model.

It can be seen that based on our designed GDMA operators, the data transfer required by AI accelerators can be more high-performance, efficient, and independent of other functional components (such as CPU), thus ensuring the overall high performance and high efficiency of the AI accelerator core.

E. TPU Subsystem Programming Model

Upon completion of TPU operator programming, the execution of these programmed operators varies based on the chosen TPU running mode, i.e. PIO (Programmed Input/Output) mode and DESC (Descriptor) mode. In PIO mode, the TPU operates in a programmed fashion, executing instructions directly based on the specified program. This mode offers a more deterministic and controlled approach, suitable for scenarios where explicit control over each instruction is essential. On the other

hand, DESC mode introduces a descriptor-based execution approach. The TPU follows a set of descriptors that define the sequence of operations to be executed. This mode enhances flexibility and is often preferred in dynamic computational scenarios where the sequence of operations might change dynamically.

In PIO mode, TPU operator programs are compiled into binary files specific to the AP Lite processor. These binary files encapsulate the essence of the TPU instructions and are disseminated to the device using specialized tools. When encountering TPU intrinsic interfaces during program execution, AP Lite processor dynamically generates TPU, GDMA, or HAU instructions in real-time. These instructions are then dispatched to the corresponding instruction queues. The TPU, GDMA, and HAU, each equipped with their dedicated instruction queues, sequentially fetch and execute the instructions residing in their respective queues. This parallel execution ensures optimal utilization of resources, enhancing overall computational efficiency. Simultaneously, the AP Lite processor serves as a scalar computing unit, enjoying direct access to DDR, L2, and local memory. This dual role empowers AP Lite to seamlessly interact with both memory and computational units, contributing to the overall synergy within the processor. In DESC mode, the TPU intrinsic interfaces called within the programs generate TPU and GDMA instructions, which are then saved as binary instruction files. The resulting binary instruction files are subsequently deployed to the DDR of the device. The TPU subsystem directly fetches instructions from the DDR space.

IV. TOOLCHAIN

The unveiling of the BM1684X architecture is accompanied by the introduction of a cutting-edge toolchain, TPU-MLIR (Tensor Processing Unit-Multi-Level Intermediate Representation). This next-generation TPU toolchain not only maintains backward compatibility with its predecessor but also integrates a suite of novel functionalities.

One of the prominent features of the TPU-MLIR toolchain is its capability to seamlessly support a multitude of neural network frameworks, including PyTorch, ONNX, TensorFlow, PaddlePaddle, and others. This inclusive approach ensures that the processing pipeline remains consistent, irrespective of the underlying framework, fostering a unified and streamlined development environment.

The TPU-MLIR toolchain adopts a layered design, abstracting the neural network model into two layers. For TOP layer, chip-agnostic operations encompasses operations such as graph optimization, quantization, and inference that are agnostic to the underlying hardware architecture. For TPU layer, chip-dependent operations involves operations specific to the TPU chip, including weight rearrangement, operator segmentation, address allocation, and chip-specific inference processes.

The overall process is shown in Fig. 17, and the model is gradually converted into the final instruction through passes. In TOP layer, shape-infer performs shape inference, including

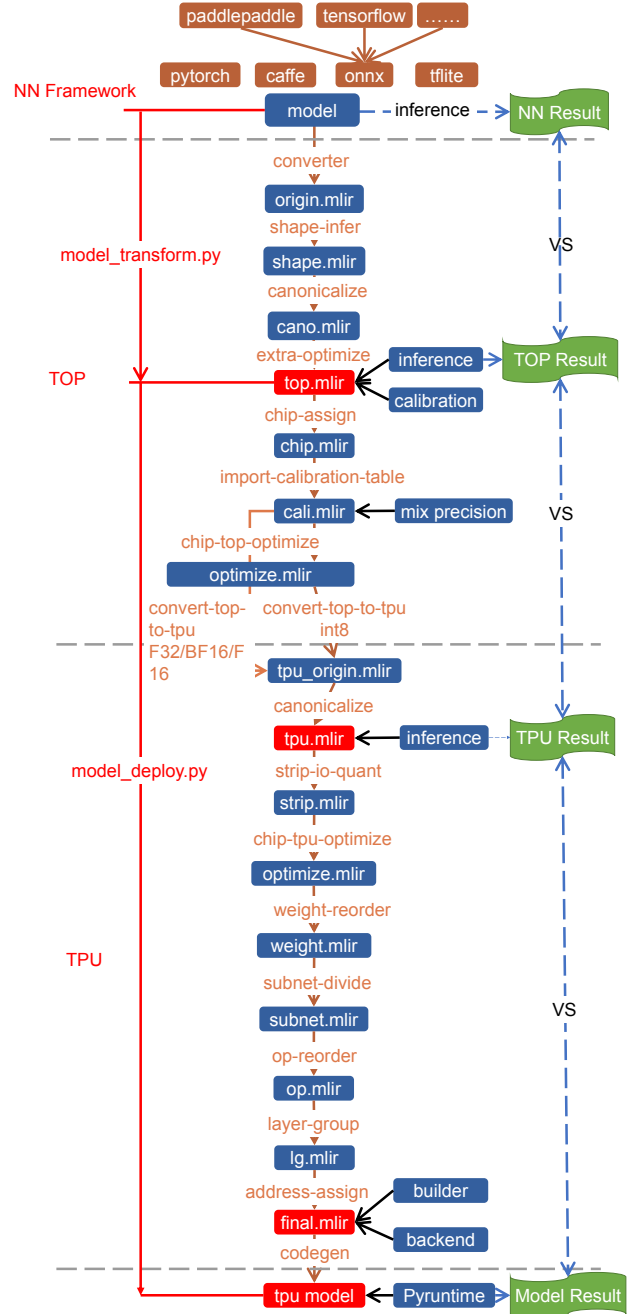


Fig. 17: TPU-MLIR.

constant folding. For an operator whose shape is uncertain, the shape is determined in this pass. Canonicalize performs graph optimization related to specific operators, such as ReLU merging to Conv, shape merge, and so on. Extra-optimize performs additional pattern implementation, such as calculating FLOPs, removal of invalid output, etc. Processor-assign configures processors, such as BM1684X. The TOP layer is adjusted according to the processor. Import-calibration-table inserts min and max into each operator according to the

calibration table for subsequent quantization. For symmetric quantization, threshold is inserted. Processor-top-optimize optimizes processor-related TOP-layer operators. Convert-top-to-tpu sinks the TOP layer to the TPU layer. If data type is floating point, the operator in the TOP layer is directly converted to the corresponding TPU layer operator. If the type is INT8, quantization conversion is required.

In TPU layer, canonicalize performs graph optimization related to TPU layer operators, such as merging of continuous requantization, and so on. Strip-io-quant decides whether the input or output is a quantized type, otherwise it is the default 32-bit floating point type. Processor-tpu-optimize performs processor-specific TPU layer operator optimization. Weight-reorder rearranges the weights of individual operators based on hardware characteristics, such as filters and bias for convolutions. Subnet-divide divides the network into different subnetworks based on processor type. If all operators are run in TPU, there is only one subnetwork. Op-reorder adjusts operator sequence. There are also special treatments for operations like attention. Layer-group slices the network so that as many operators as possible are continuously computed in the local memory. Address-assign assigns an address to the operator that requires global memory. Codegen executes the codegen interface of operator to generate command buffer. The final model is generated using the builder module in the format of FlatBuffers.

To guarantee the correctness of the TPU-MLIR toolchain, a meticulous approach is taken. The inference results from the top layer are compared against ONNX results, ensuring alignment and consistency throughout the model processing. The inference results from the TPU layer are cross-verified against the top layer, establishing the precision and reliability of the TPU-specific operations. The inference results from the TPU layer are rigorously compared with the results obtained from the Cmodel, affirming the hardware consistency.

The TPU-MLIR toolchain offers a unique advantage by generating MLIR at each transformation step. This facilitates effective tracking and debugging throughout the entire model transformation process, enhancing the toolchain's transparency and facilitating efficient debugging. Our solution is like the one in [1], [2], [21].

V. EVALUATION

We position our processor as high-performance and versatile AI solutions suitable for processing applications such as natural language processing [14] and computer vision [57]. Therefore, our tests are mainly conducted on AI models such as BERT, ResNet18, ResNet50 and Mobilenet.

A. Comparison with Other Chips

In the comparison of AI processor performance, we focus on two prominent AI models, BERT and ResNet50, using latency data as a metric for evaluation. Latency data represents the time required for a chip to process input data and generate model output during inference tasks. Smaller latency values are preferred, as they reflect faster response times in real-world

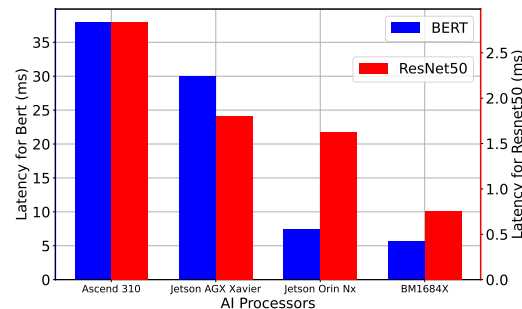


Fig. 18: Latency of different processors.

applications. Note that, the data in the evaluation is gathered in a real system. And the exact same application configurations are run on all the competing designs. All the comparisons are based on actual test results of publicly available products. Due to the differences in architectures, programming models, and instruction sets, different AI accelerators use different software stacks for inference and calculation of the same AI model, but we select products with the same computing power and DRAM bandwidth in the comparison, we adopt the same test cases, and we compile each AI model to machine codes by using the latest and optimized software stacks of various manufacturers.

For the selected models, we conducted latency tests on four AI processors: Ascend 310, Jetson AGX Xavier, Jetson Orin NX 8GB, and BM1684X. Fig. 18 shows the latency performance. The obtained data reveals that BM1684X exhibits impressive latency performance, with a latency of 6 for the BERT model and 1 for the ResNet50 model. In contrast, the other three chips show relatively higher latencies in the same tasks.

The comparison of latency data highlights the superior performance of BM1684X in both the BERT and ResNet50 models, showing significantly shorter response times. This is particularly advantageous for real-time applications and tasks with stringent latency requirements. Consequently, BM1684X demonstrates a pronounced advantage in latency performance, providing an efficient inference solution across diverse applications.

In offline or multi-batch evaluation, two AI models, BERT and ResNet50, are still used to compare with Ascend 310, Jetson AGX Xavier and Jetson Orin NX 8GB. The presented data shown in Fig. 19 focuses on the inference performance of the BERT model across four AI processors: Ascend 310, Jetson AGX Xavier, Jetson Orin NX 8GB, and BM1684X. Two critical metrics, "Performance/Computing Power" and "Performance/Bandwidth" have been examined to evaluate the efficiency and effectiveness of each processor in handling BERT model inferences, where performance refers to throughput in units of frames/second. The "Performance/Computing Power" ratio gauges how well each AI chip utilizes its theoretical computing power for BERT model inference. BM1684X emerges as the clear leader in this aspect with a notable ratio of 6.87. This high ratio underscores BM1684X's exceptional efficiency

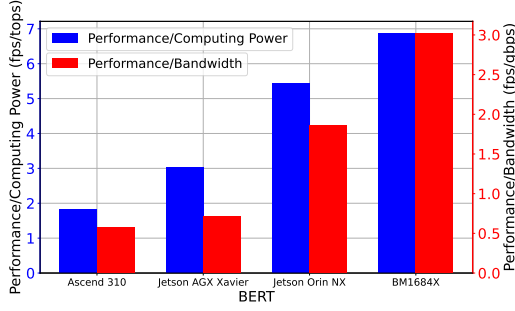


Fig. 19: Inference performance for BERT model.

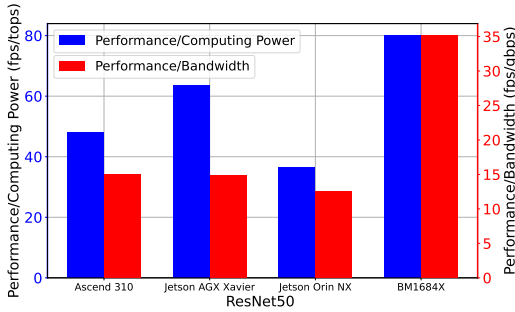


Fig. 20: Inference performance for ResNet50 model.

in translating theoretical computing power into tangible inference performance. Similarly, the “Performance/Bandwidth” ratio assesses how efficiently each AI processor leverages its theoretical bandwidth to achieve BERT model inference performance. Once again, BM1684X demonstrates remarkable efficiency with a ratio of 3.02, outperforming its counterparts. This result suggests that BM1684X optimally utilizes available bandwidth, showing its ability to deliver superior performance.

Fig. 20 examines the ResNet50 model’s inference performance across four AI processors. When considering “Performance/Computing Power”, BM1684X outshines its competitors, boasting an outstanding ratio of 79.97. This surpasses the ratios for Ascend 310 (48.00), Jetson AGX Xavier (63.72), and Jetson Orin NX 8GB (36.60). BM1684X’s commanding lead in this metric underscores its exceptional efficiency in translating theoretical computing power into tangible ResNet50 model inference performance. Turning our attention to the “Performance/Bandwidth” ratio, BM1684X once again takes the lead with a ratio of 35.12. This highlights its superior effectiveness in utilizing theoretical bandwidth for ResNet50 model inferences. This result outpaces the ratios for Ascend 310 (15.00), Jetson AGX Xavier (14.88), and Jetson Orin NX 8GB (12.51). BM1684X’s optimized use of available bandwidth reinforces its ability to deliver exceptional ResNet50 model inference performance compared to its counterparts.

Our chip applies DSA (Domain Specific Accelerator), while based on Nvidia’s public information, the competitive Jetson Orin NX 8GB applies GPGPU+DSA. Jetson Orin NX 8GB which offers a maximum computing power of 35 TOPS@INT8

includes about 25 TOPS from GPGPU and 10 TOPS from DSA NVDLA for AI acceleration, while all the computing power of our chip comes from DSA TPU for AI acceleration. The main reason DSAs designed for deterministic applications have higher computing power utilization, cost performance, and energy efficiency ratio than GPGPU is that, from an architectural perspective: (1) a single SIMD (Single Instruction Multiple Data) core architecture with very large data width has lower overhead than multiple SIMT (Single Instruction Multiple Thread) cores which work together by maintaining cache coherence and synchronizing messages; (2) for calculating AI models, a single architecture (DSA) has lower overhead than heterogeneous collaborated architectures (GPGPU+DSA), due to avoiding data sharing and message synchronization between the processors of the two architectures; (3) for calculating AI models, the DMA-based prefetch mechanism has higher cost performance and energy efficiency ratio than SIMT, due to avoiding complex memory access mechanisms and instruction re-issue; (4) for CNN applications, designing high-performance convolutional special instructions can greatly improve performance, and the key to support convolutional special instructions is to have a data permute logic for IMG2COL which is synchronously executed with matrix multiplication, which is the Crossbar module in datapath and related control logic in BM1684X. In GPGPU and the DSAs of competing products, due to the lack of this dedicated IMG2COL design implementation, IMG2COL needs to be implemented using a Load command which has low performance, and IMG2COL results require substantial additional cache.

B. Comparison under Different Batch Sizes

We take performance evaluations with popular AI models, including mobilenet, ResNet18, and ResNet50, across various batch sizes. The objective is to comprehensively assess the product’s efficiency under different workloads. The normalized throughput data, representing the unit-time processing capability under respective computational powers, facilitates a clearer assessment of the product’s adaptability and efficiency under different operational scenarios, unveils insightful performance trends. The results are shown in Fig. 21.

BM1684X exhibits a consistent improvement in throughput as the batch size increases. The performance gain is particularly notable, reaching almost 378% improvement at batch size 32 compared to batch size 1. This suggests that BM1684X excels in handling ResNet18 model, demonstrating scalability with increased batch sizes. Similar to ResNet18, BM1684X shows a steady increase in performance with larger batch sizes. The throughput at batch size 32 is approximately 389% higher than at batch size 1. This consistency in improvement across batch sizes indicates BM1684X’s capability to efficiently process ResNet50 model. For Mobilenet model, BM1684X again demonstrates a commendable performance boost with growing batch sizes. The throughput at batch size 32 registers an approximately 336% improvement compared to batch size 1. This signifies the adaptability

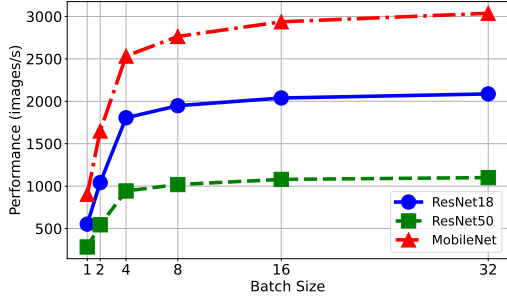


Fig. 21: Performance of BM1684X in classic AI network inference.

of BM1684X across different neural network architectures, including lightweight models like MobileNet.

C. Inference Performance on Large Models

BM1684X exhibits compelling performance across diverse large models, with table III showing its versatility and efficiency in various domains.

The general-purpose models, chatglm2-6b and chatglm-6b, demonstrate robust token processing capabilities, achieving 7 tokens/s and 11 tokens/s in INT8 and INT4, respectively. Notably, chatglm-6b exhibits a remarkable performance of 7 INT8 tokens/s, emphasizing its efficiency in token-based tasks. In language models, llama2-7b and Qwen-7b shows BM1684X's prowess in language-oriented tasks. llama2-7b operates at 5 tokens/s in INT8 and 9 tokens/s in INT4, while Qwen-7b achieves a performance of 9 tokens/s in INT4, affirming its suitability for language processing applications. Vision models, including stablediffusion-v1.5, sdxl and stablediffusion-v2.1, exhibit impressive efficiency with a processing time of 0.31s/step and 1.7s/step, respectively. These results underscore BM1684X's competence in handling vision-related tasks. In multi-modal applications, clip stands out by efficiently building an index for 1000 images in 9s and executing single queries on a database of 2 million images in just 4ms. This demonstrates BM1684X's excellence in both image and video search applications. For speech-related tasks, whisper efficiently transcribes 5 minutes of speech in 1 minute, while emotivoice demonstrates rapid text-to-speech conversion, processing 110s of audio in a mere 3s. Specialized models, such as codegeex2, real-ESRGAN, and sadtalker, highlight BM1684X's efficiency in specific domains. Codegeex2 specializes in code generation with a performance of 9 tokens/s in INT4, Real-ESRGAN processes 10 frames per second for super-resolution tasks, and sadtalker completes the entire process for a ten-second video in 40s, addressing the domain of digital personalities.

Our SoC supports training. Since our original design purpose is inference, so the training performance is not high due to its design in computing power, DRAM bandwidth, multi-core interconnection. In typical application scenarios, users can use our processor to fine-tune model parameters locally

TABLE III: Inference Performance on Large Models.

Model	Performance	Category
Chatglm2-6b	INT8 7 tokens/s INT4 11 tokens/s	universal
Chatglm-6b	INT8 7 tokens/s	universal
llama2-7b	INT8 5 tokens/s INT4 9 tokens/s	universal
Qwen-7b	INT4 9 tokens/s	universal
Stable diffusion v1.5	0.31s/step	text to picture
Sdxl	1.7s/step	text to picture
Stablediffusion v2.1	0.31s/step	text to picture
Clip	indexing 1000 pictures spends 9s database with 200 thousand items querying top100 takes 4ms	text search picture or video
whisper	5 mins of speech per min	speech to text
Emotivoice	110s audio takes 3s	text to speech
Codegeex2	INT4 9 tokens/s	code generation
Real-ESRGAN	10 fps	super resolution
Sadtalker	10s'video takes 40s for entire process	digital man

using the latest local labeled data, and then deploy the fine-tuned new model, instead of uploading the latest local labeled data to the cloud for training.

VI. RELATED WORK

Numerous studies and papers have categorized various cards and chips according to their intended applications, primarily considering performance and power consumption [12], [20], [22], [26], [32], [39], [46]. The classifications are as follows: Sensors for voice processing that are characterized by extremely low power consumption and very small size [9], [47]; Embedded processors that demand high performance relative to power and form factor, utilized in camera processors, small drones, and robots [3], [27]; Inference processors tailored for automotive AI/ML, autonomous vehicles, UAVs, robots, etc. [10], [13], [19], [23], [31], [50]; Chips and cards for data centers, which typically include multiple CPUs, GPUs, programmable FPGA solutions, and programmable accelerators [11], [24], [28], [35]–[37], [40], [44], [48], [53]; and data center systems [4], [16].

Among AI accelerators for edge computing [29], [33], [38], [52], we pay attention to the AI chips of Huawei and Nvidia. Huawei Ascend 310 is an AI chip that outputs 16TOPS@INT8, 8TOPS@FP16 and supports 16-channel full HD video decoding in typical configurations. The maximum power consumption is 8W. It adopts Huawei Da Vinci architecture, integrates rich computing units, improves the completeness and efficiency of AI calculation, and then expands the applicability of the chip [8]. The Jetson AGX Xavier chip is composed of NVIDIA's Carmel architecture with 8-core 64-bit CPU and Volta architecture with 512 CUDA processor GPU (single-precision floating-point performance of 2.8 TFLOPS, double-precision 1.4 TFLOPS) and is equipped with a deep learning accelerator with an INT8 computing performance of up to 32 TOPS, as well as a programmable vision accelerator [45]. The Jetson Orin NX offers a maximum computing power of 35 TOPS and a maximum peak power

of 20W, and has end-to-end application pipeline acceleration. It is suitable for small and low-power autonomous machines such as robots and factory automation [15]. Our BM1684X and selected competitors are SOC's which are edge computing chips. Since our BM1684X targets to terminal chip for edge computing with power consumption not exceeding 40W, we make comparison with Huawei Ascend 310, Jetson AGX Xavier and Jetson Orin NX.

VII. CONCLUSION AND LESSONS LEARNED

The paper introduces BM1684X, an advanced AI processor developed by SOPHGO to meet the diverse requirements of a wide range of AI applications. The TPU of BM1684X is designed with an SIMD architecture featuring very large data width to reduce the area ratio of the instruction unit and enhance computing power density. Special acceleration instructions within EU enable dynamic pipeline execution, leading to a reduction in the total number of instructions and execution time, thereby enhancing the performance of the TPU in processing RQ and DQ operations. The TPU's CUBE array executes multiplication and addition operations of 64 pairs of INT8 operands in the channel dimension of the feature map. By utilizing an addition tree instead of a conventional adder, this implementation significantly reduces area and power consumption, optimizing TPU efficiency. BM1684X incorporates a 64-input, 64-output, 8-bit crossbar within the lane, enabling high-performance data gathering and enhancing data gathering capabilities for efficient processing within the TPU architecture. BM1684X offers three memory access modes to demonstrate flexibility in catering to diverse AI processing requirements and optimizing DRAM utilization for different tasks and workloads. The design of a TPU-MLIR toolchain is highlighted, showing features such as unified processing of multiple frameworks, hierarchical model abstractions, correctness guarantees, and traceability of transformation steps. BM1684X demonstrates exceptional performance in delivering high-performance computing for a variety of AI models, including large models. This is evidenced through comprehensive evaluations conducted alongside industry-leading peers, showing the processor's capabilities in handling complex and demanding AI workloads efficiently and effectively.

Comparing our TPU with competing products, it can be learned that the computing power utilization or efficiency of our TPU is higher than that of GPGPU+NVDLA used in Nvidia Jetson. For CNN networks such as ResNet50, the main operator is convolution. Jetson does not provide a separate circuit support for this operator, but uses IMG2COL (image-to-column) to expand the convolved image to matrix, but the efficiency is not high. In contrast, our TPU is designed with a separate circuit to support it, so it has higher performance under the same computing power and DRAM bandwidth. For Transformer networks such as BERT, such network operators are relatively simple, and our TPU has the same performance with Nvidia Jetson.

ACKNOWLEDGMENT

We thank the anonymous reviewers for their insightful and constructive feedback.

REFERENCES

- [1] TPU-MLIR, <https://tpumlr.org/>.
- [2] TPU-MLIR Technical Reference Manual, https://doc.sophgo.com/sdk-docs/v23.09.01-lts/docs_latest_release/docs/tpu-mlir/developer_manual_en/html/index.html.
- [3] M. S. Abdelfattah, D. Han, A. Bitar, R. DiCecco, S. O'Connell, N. Shanker, J. Chu, I. Prins, J. Fender, A. C. Ling, and G. R. Chiu, "DLA: Compiler and FPGA Overlay for Neural Network Inference Acceleration," in *Proceedings of 28th International Conference on Field Programmable Logic and Applications (FPL)*, 2018, pp. 4110–4117.
- [4] D. Abts, J. Ross, J. Sparling, M. Wong-VanHaren, M. Baker, T. Hawkins, A. Bell, J. Thompson, T. Kahsai, G. Kimmell, J. Hwang, R. Leslie-Hurd, M. Bye, E. Creswick, M. Boyd, M. Venigalla, E. Laforge, J. Purdy, P. Kamath, D. Maheshwari, M. Beidler, G. Rosseel, O. Ahmad, G. Gagarin, R. Czekalski, A. Rane, S. Parmar, J. Werner, J. Sproch, A. Macias, and B. Kurtz, "Think Fast: A Tensor Streaming Processor (TSP) for Accelerating Deep Learning Workloads," in *Proceedings of ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*, 2020, pp. 145–158.
- [5] M. Alam, M. D. Samad, L. Vidyaratne, A. Glandon, and K. M. Iftekharuddin, "Survey on Deep Neural Networks in Speech and Vision Systems," *Neurocomputing*, vol. 417, pp. 302–321, 2020.
- [6] F. Ashtiani, A. J. Geers, and F. Aflatouni, "An On-chip Photonic Deep Neural Network for Image Classification," *Nature*, vol. 606, no. 7914, pp. 501–506, 2022.
- [7] W. Bao, Q. Yu, and Y. Kong, "Evidential Deep Learning for Open Set Action Recognition," in *Proceedings of the IEEE International Conference on Computer Vision*, 2021, pp. 13 349–13 358.
- [8] C. Chen, X. Xiang, C. Liu, Y. Shang, R. Guo, D. Liu, Y. Lu, Z. Hao, J. Luo, Z. Chen, C. Li, Y. Pu, J. Meng, Y. Xiaolang, Y. Xie, and X. Qi, "Xuantie-910: A Commercial Multi-core 12-stage Pipeline Out-of-order 64-bit High Performance RISC-V Processor with Vector Extension: Industrial Product," in *Proceedings of the 47th Annual International Symposium on Computer Architecture (ISCA)*, 2020, pp. 52–64.
- [9] Y. Chen, Y. Xie, L. Song, F. Chen, and T. Tang, "A Survey of Accelerator Architectures for Deep Neural Networks," *Engineering*, vol. 6, no. 3, pp. 264–274, 2020.
- [10] Y.-H. Chen, T. Krishna, J. S. Emer, and V. Sze, "Eyeriss: An Energy-Efficient Reconfigurable Accelerator for Deep Convolutional Neural Networks," *IEEE Journal of Solid-State Circuits*, vol. 52, no. 1, pp. 127–138, 2017.
- [11] Y. Chen, T. Luo, S. Liu, S. Zhang, L. He, J. Wang, L. Li, T. Chen, Z. Xu, and N. Sun, "DaDianNao: A Machine-Learning Supercomputer," in *Proceedings of 47th Annual IEEE/ACM International Symposium on Microarchitecture (Micro)*, 2014, pp. 609–622.
- [12] C. Chris Duckett, "Baidu Creates Kunlun Silicon for AI," 2018, <https://www.zdnet.com/article/baidu-creates-kunlun-silicon-for-ai/>.
- [13] C. Evangelist, "Deep Dive into Amazon Inferentia: A Custom-Built Chip to Enhance ML and AI," 2020, <https://www.cloudmanagementinsider.com/amazon-inferentia-for-machine-learning-and-artificial-intelligence/>.
- [14] S. Y. Feng, V. Gangal, J. Wei, S. Chandar, S. Vosoughi, T. Mitamura, and E. Hovy, "A Survey of Data Augmentation Approaches for NLP," 2021, <https://arxiv.org/abs/2105.03075>.
- [15] B. Funk, "NVIDIA Jetson AGX Orin: The Next-Gen Platform That Will Power Our AI Robot Overlords Unveiled," 2022, <https://hothardware.com/news/nvidia-jetson-agx-orin>.
- [16] M. Gao, J. Pu, X. Yang, M. Horowitz, and C. Kozyrakis, "TETRIS: Scalable and Efficient Neural Network Acceleration with 3D Memory," *Proceedings of the Twenty-Second International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, pp. 751–764, 2017.
- [17] Y. Gong, M. Yin, L. Huang, J. Xiao, Y. Sui, C. Deng, and B. Yuan, "ETTE: Efficient Tensor-Train-based Computing Engine for Deep Neural Networks," in *Proceedings of the 50th Annual International Symposium on Computer Architecture (ISCA)*, 2023, pp. 1–13.

- [18] A. Gonzalez, A. Kolli, S. Khan, S. Liu, V. Dadu, S. Karandikar, J. Chang, K. Asanovic, and P. Ranganathan, "Profiling Hyperscale Big Data Processing," in *Proceedings of the 50th Annual International Symposium on Computer Architecture (ISCA)*, 2023, pp. 1–16.
- [19] J. Hamilton, "AWS Inferentia Machine Learning Processor," 2018, <https://perspectives.mvdirona.com/2018/11/aws-inferentia-machine-learning-processor/>.
- [20] J. L. Hennessy and D. A. Patterson, "A New Golden Age for Computer Architecture," *Communications of the ACM*, vol. 62, no. 2, p. 4860, 2019.
- [21] P. Hu, M. Lu, L. Wang, and G. Jiang, "TPU-MLIR: A Compiler For TPU Using MLIR," 2023, <http://arxiv.org/abs/2210.15016>.
- [22] C. D. James, J. B. Aimone, N. E. Miner, C. M. Vineyard, F. H. Rothganger, K. D. Carlson, S. A. Mulder, T. J. Draelos, A. Faust, M. J. Marinella, J. H. Naegle, and S. J. Plimpton, "A Historical Survey of Algorithms and Hardware Architectures for Neural-inspired and Neuromorphic Computing Applications," *Biologically Inspired Cognitive Architectures*, vol. 19, pp. 49–64, 2017.
- [23] Y. Jiao, L. Han, and X. Long, "Hanguang 800 NPU - The Ultimate AI Inference Solution for Data Centers," *2020 IEEE Hot Chips 32 Symposium (HCS)*, pp. 1–29, 2020.
- [24] N. P. Jouppi, D. H. Yoon, G. Kurian, S. Li, N. Patil, J. Laudon, C. Young, and D. Patterson, "A Domain-Specific Supercomputer for Training Deep Neural Networks," *Communications of the ACM*, vol. 63, no. 7, pp. 67–78, 2020.
- [25] R. Ke, A. Bugeau, N. Papadakis, M. Kirkland, P. Schuetz, and C.-B. Schlieb, "Multi-Task Deep Learning for Image Segmentation Using Recursive Approximation Tasks," *IEEE Transactions on Image Processing*, vol. 30, pp. 3555–3567, 2021.
- [26] S. Khan, "AI Chips: What They Are and Why They Matter," 2020, <https://cset.georgetown.edu/publication/ai-chips-what-they-are-and-why-they-matter/>.
- [27] M. Kim, J. Kim, G. Park, L. Everson, H. Kim, S. Song, S. Lee, and C. H. Kim, "A 68 Parallel Row Access Neuromorphic Core with 22K Multi-Level Synapses Based on Logic-Compatible Embedded Flash Memory Technology," in *Proceedings of IEEE International Electron Devices Meeting (IEDM)*, 2018, pp. 1–4.
- [28] R. Krashinsky, O. Giroux, S. Jones, N. Stam, and S. Ramaswamy, "NVIDIA Ampere Architecture In-Depth," 2020, <https://developer.nvidia.com/blog/nvidia-ampere-architecture-in-depth/>.
- [29] H. F. Langroudi, T. Pandit, M. Indovina, and D. Kudithipudi, "Digital Neuromorphic Chips for Deep Learning Inference: A Comprehensive Study," in *Proceedings of Applications of Machine Learning*, 2019, p. 9.
- [30] J. Lee, P. Wang, R. Xu, S. Jain, V. Dasari, N. Weston, Y. Li, S. Bagchi, and S. Chaterji, "Virtuoso: Energy- and Latency-aware Streamlining of Streaming Videos on Systems-on-Chips," *ACM Transactions On Design Automation Of Electronic Systems*, vol. 28, no. 31, pp. 1–32, 2023.
- [31] S. Liu, Z. Du, J. Tao, D. Han, T. Luo, Y. Xie, Y. Chen, and T. Chen, "Cambricon: An Instruction Set Architecture for Neural Networks," 2016, pp. 393–405.
- [32] R. Merritt, "Baidu Accelerator Rises in AI," 2018, <https://www.eetimes.com/baidu-accelerator-rises-in-ai/>.
- [33] J. Misra and I. Saha, "Artificial Neural Networks in Hardware: A Survey of Two Decades of Progress," *Neurocomputing*, vol. 74, pp. 1239–255, 2010.
- [34] Y. Mo, Y. Wu, X. Yang, F. Liu, and Y. Liao, "Review the State-of-the-art Technologies of Semantic Segmentation Based on Deep Learning," *Neurocomputing*, vol. 493, pp. 626–646, 2022.
- [35] T. P. Morgan, "Nvidia Rounds Out 'Ampere' Lineup With Two New Accelerators," 2020, <https://www.nextplatform.com/2021/04/15/nvidia-rounds-out-ampere-lineup-with-two-new-accelerators/>.
- [36] J. Ouyang, X. Du, Y. Ma, and J. Liu, "Kunlun: A 14nm High-Performance AI Processor for Diversified Workloads," in *Proceedings of IEEE International Solid-State Circuits Conference (ISSCC)*, 2021, pp. 50–51.
- [37] O. Peckham, "Google Cloud's New TPU v4 ML Hub Packs 9 Exaflops of AI," 2022, <https://www.hpcwire.com/2022/05/16/google-clouds-new-tpu-v4-ml-hub-packs-9-exaflops-of-ai/>.
- [38] A. Reuther, P. Michaleas, M. Jones, V. Gadepally, S. Samsi, and J. Kepner, "Survey and Benchmarking of Machine Learning Accelerators," in *Proceedings of IEEE High Performance Extreme Computing Conference (HPEC)*, 2019, pp. 1–9.
- [39] T. Rogers and M. Khairy, "An Academic's Attempt to Clear the Fog of the Machine Learning Accelerator War," 2021, <https://www.sigarch.org/an-academics-attempt-to-clear-the-fog-of-the-machine-learning-accelerator-war/>.
- [40] G. Roos, "FPGA Acceleration Card Delivers on Bandwidth, Speed, and Flexibility," 2019, <https://www.eetimes.com/fpga-acceleration-card-delivers-on-bandwidth-speed-and-flexibility/>.
- [41] C. Saharia, W. Chan, S. Saxena, L. Li, J. Whang, E. L. Denton, K. Ghasemipour, R. Gontijo Lopes, B. Karagol Ayan, T. Salimans *et al.*, "Photorealistic Text-to-image Diffusion Models with Deep Language Understanding," *Advances in Neural Information Processing Systems*, vol. 35, pp. 36 479–36 494, 2022.
- [42] H. Salehinejad, S. Sankar, J. Barfett, E. Colak, and S. Valaee, "Recent Advances in Recurrent Neural Networks," 2018, <https://arxiv.org/abs/1801.01078>.
- [43] J. S. Smith, P. Cascante-Bonilla, A. Arbelles, D. Kim, R. Panda, D. Cox, D. Yang, Z. Kira, R. Feris, and L. Karlinsky, "ConStruct-VL: Data-Free Continual Structured VL Concepts Learning," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023, pp. 14 994–15 004.
- [44] R. Smith, "16GB NVIDIA Tesla V100 Gets Reprieve; Remains in Production," 2018, <https://www.anandtech.com/show/12809/16gb-nvidia-tesla-v100-gets-reprieve-remains-in-production>.
- [45] —, "NVIDIA Gives Jetson AGX Xavier a Trim, Announces Nano-Sized Jetson Xavier NX," 2019, <https://www.anandtech.com/show/15070/nvidia-gives-jetson-xavier-a-trim-announces-nanosized-jetson-xavier-nx>.
- [46] F. P. Sunny, E. Taheri, M. Nikdast, and S. Pasricha, "A Survey on Silicon Photonics for Deep Learning," *ACM Journal on Emerging Technologies in Computing Systems*, vol. 17, no. 61, pp. 1–57, 2021.
- [47] V. Sze, Y.-H. Chen, T.-J. Yang, and J. S. Emer, "Efficient Processing of Deep Neural Networks: A Tutorial and Survey," 2017, <https://arxiv.org/abs/1703.09039>.
- [48] E. Talpes, D. D. Sarma, G. Venkataramanan, P. Bannon, B. McGee, B. Floering, A. Jalote, C. Hsiong, S. Arora, A. Gorti, and G. S. Sachdev, "Compute Solution for Tesla's Full Self-Driving Computer," *IEEE Micro*, vol. 40, no. 2, pp. 25–35, 2020.
- [49] T. Tambe, E.-Y. Yang, G. G. Ko, Y. Chai, C. Hooper, M. Donato, P. N. Whatmough, A. M. Rush, D. Brooks, and G.-Y. Wei, "A 16-nm SoC for Noise-Robust Speech and NLP Edge AI Inference With Bayesian Sound Source Separation and Attention-Based DNNs," *IEEE Journal of Solid-State Circuits*, vol. 58, no. 2, pp. 569–581, 2023.
- [50] P. Teich, "Tearing Apart Google's TPU 3.0 AI Coprocessor," 2018, <https://www.nextplatform.com/2018/05/10/tearing-apart-googles-tpu-3-0-ai-coprocessor/>.
- [51] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention Is All You Need," 2023, <http://arxiv.org/abs/1706.03762>.
- [52] E. Wang, J. J. Davis, R. Zhao, H.-C. Ng, X. Niu, W. Luk, P. Y. Cheung, and G. A. Constantinides, "Deep Neural Network Approximation for Custom Hardware: Where We've Been, Where We're Going," *ACM Computing Surveys*, vol. 52, no. 40, p. 139, 2019.
- [53] S. Ward-Foxton, "Graphcore Takes on Nvidia with Second-Gen AI Accelerator," 2020, <https://www.eetimes.com/graphcore-takes-on-nvidia-with-second-gen-ai-accelerator/>.
- [54] J. Yang, H. Jin, R. Tang, X. Han, Q. Feng, H. Jiang, B. Yin, and X. Hu, "Harnessing the Power of LLMs in Practice: A Survey on ChatGPT and Beyond," 2023, <https://arxiv.org/abs/2304.13712>.
- [55] Z. Ying, S. Bhuyan, Y. Kang, Y. Zhang, M. T. Kandemir, and C. R. Das, "EdgePC: Efficient Deep Learning Analytics for Point Clouds on Edge Devices," in *Proceedings of the 50th Annual International Symposium on Computer Architecture (ISCA)*, 2023, pp. 1–14.
- [56] S. Zhang, L. Wen, X. Bian, Z. Lei, and S. Z. Li, "Single-Shot Refinement Neural Network for Object Detection," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 4203–4212.
- [57] S. Zhang, Z. Wei, W. Xu, L. Zhang, Y. Wang, J. Zhang, and J. Liu, "Edge Aware Depth Inference for Large-Scale Aerial Building Multi-view Stereo," *ISPRS Journal of Photogrammetry and Remote Sensing*, vol. 207, pp. 27–42, 2024.

APPENDIX

A. Our proposed EU

B. ucode and data flow of floating-point division

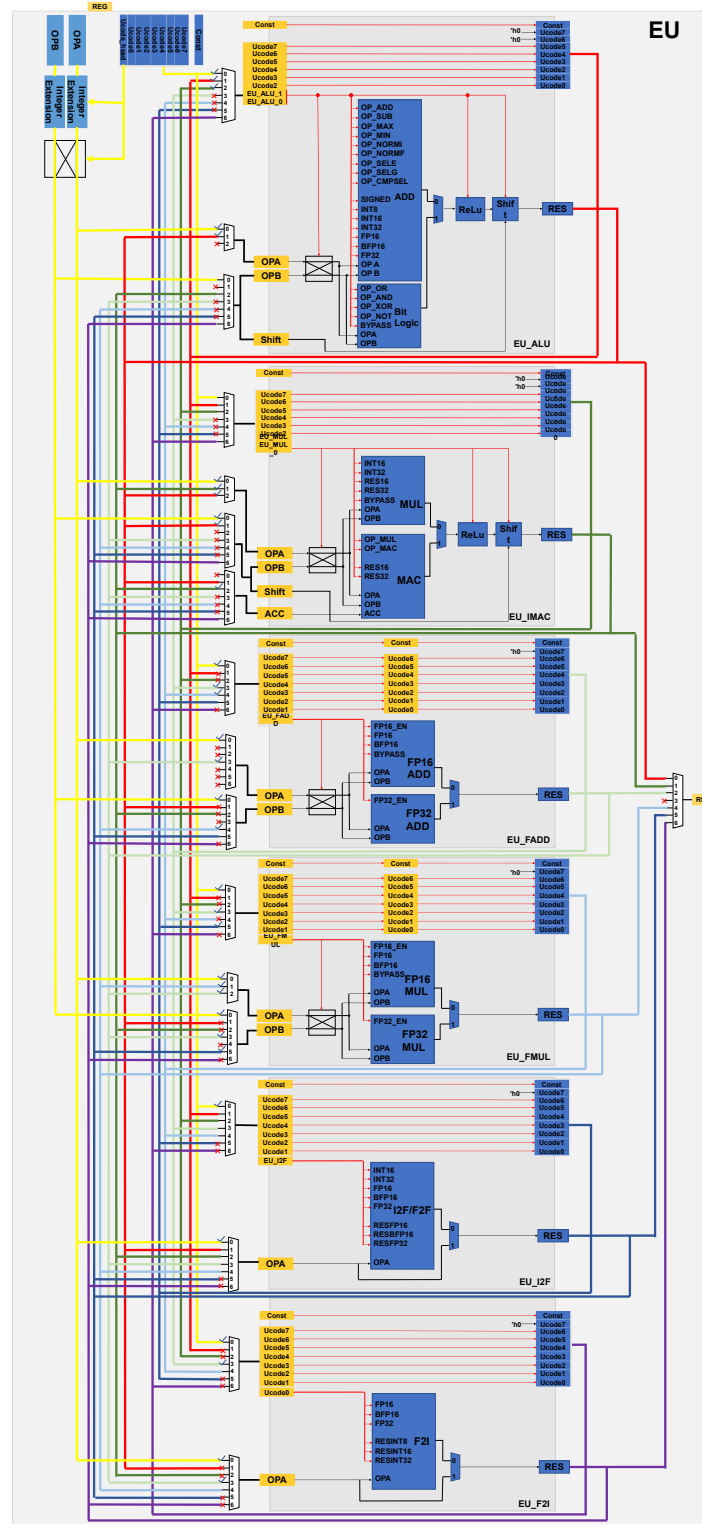


Fig. 22: Our proposed EU.

	FMUL				FADD		ALU		RESULT	UCODE0	UCODE1	UCODE2	UCODE3	UCODE4
	OPA	OPB	OPC	OPD	OPA	OPB	OPA	OPB		EU_HEAD[OPX]				
cyc0	NU	NU	B(0, 0, 0, 0~1)	A(0, 0, 0, 0~1)	NU	NU	B(0, 0, 0, 0~1)	NU		EU_FMUL[UC UALUA C*0.5F]				
cyc1	NU	NU	NU	A(0, 0, 0, 0~2)	NU	NU	NU	NU		EU_FMUL_DLY	EU_ALU[OPX MAGIC_SUB]			
cyc2	RP0-(0, 0, 0, 0~1)	NU	RP0X(0, 0, 0, 0~1)	A(0, 0, 0, 0~3)	NU	NU	NU	NU		EU_MUL[UC]	EU_FMUL[UA]	EU_FMUL[A*C]		
cyc3	RP0-(0, 0, 0, 0~1)	NU	RP0X(0, 0, 0, 0~1)	A(0, 0, 0, 0~4)	NU	NU	NU	NU		EU_END	EU_END	EU_FMUL_ DLY		
cyc4	RP0-(0, 0, 0, 0~1)	RP1X(0, 0, 0, 0~1)	RP0X(0, 0, 0, 0~1)	A(0, 0, 0, 0~5)	NU	NU	NU	NU				EU_FMUL[UB A*B]		
cyc5	RP0-(0, 0, 0, 0~1)	NU	RP0X(0, 0, 0, 0~1)	A(0, 0, 0, 0~6)	NU	NU	NU	NU				EU_FMUL_ DLY		
cyc6	RP0-(0, 0, 0, 0~1)	NU	RP0X(0, 0, 0, 0~1)	A(0, 0, 0, 0~7)	NU	RP2X(0, 0, 0, 0~1)	NU	NU				EU_FADD[UB 1.5F-B]		
cyc7	RP0-(0, 0, 0, 0~1)	NU	RP0X(0, 0, 0, 0~1)	A(0, 0, 0, 0~8)	NU		NU	NU				EU_FADD_ DLY		
cyc8	RP0-(0, 0, 0, 0~1)	RP2-(0, 0, 0, 0~1)	RP0X(0, 0, 0, 0~1)	A(0, 0, 0, 0~9)	NU		NU	NU				EU_FMUL[UB A*B]		
cyc9	NU	NU	RP0X(0, 0, 0, 0~1)	A(0, 0, 0, 0~10)	NU		NU	NU				EU_FMUL_ DLY		
cyc10	RP3X(0, 0, 0, 0~1)	NU	RP0X(0, 0, 0, 0~1)	A(0, 0, 0, 0~11)	NU		NU	NU				EU_FMUL[UA]	EU_FMU L[A*C]	
cyc11	RP3X(0, 0, 0, 0~1)	NU	RP0X(0, 0, 0, 0~1)	A(0, 0, 0, 0~12)	NU		NU	NU				EU_FMU L_DLY		
cyc12	RP3X(0, 0, 0, 0~1)	RP4X(0, 0, 0, 0~1)	RP0X(0, 0, 0, 0~1)	A(0, 0, 0, 0~13)	NU		NU	NU				EU_FMU L[UB A*B]		
cyc13	RP3X(0, 0, 0, 0~1)	NU	RP0X(0, 0, 0, 0~1)	A(0, 0, 0, 0~14)	NU		NU	NU				EU_FMU L_DLY		
cyc14	RP3X(0, 0, 0, 0~1)	NU	RP0X(0, 0, 0, 0~1)	A(0, 0, 0, 0~15)	NU	RP5X(0, 0, 0, 0~1)	NU	NU				EU_FAD D[UB 1.5F-B]		
cyc15	RP3X(0, 0, 0, 0~1)	NU	RP0X(0, 0, 0, 0~1)	A(0, 0, 0, 0~16)	NU		NU	NU				EU_FAD D_DLY		
cyc16	RP3X(0, 0, 0, 0~1)	RP5-(0, 0, 0, 0~1)	RP0X(0, 0, 0, 0~1)	A(0, 0, 0, 0~17)	NU		NU	NU				EU_FMU L[UB A*B]		
cyc17	NU	NU	RP0X(0, 0, 0, 0~1)	A(0, 0, 0, 0~18)	NU		NU	NU				EU_FMU L_DLY		
cyc18	RP6X(0, 0, 0, 0~1)	NU	RP0X(0, 0, 0, 0~1)	A(0, 0, 0, 0~19)	NU		NU	NU				EU_FMU L[UA]	EU_FMUL[A*C]	
cyc19	RP6X(0, 0, 0, 0~1)	NU	RP0X(0, 0, 0, 0~1)	A(0, 0, 0, 0~20)	NU		NU	NU				EU_FMU L_DLY		
cyc20	RP6X(0, 0, 0, 0~1)	RP7X(0, 0, 0, 0~1)	RP0X(0, 0, 0, 0~1)	A(0, 0, 0, 0~21)	NU		NU	NU				EU_FMU[UB A*B]		
cyc21	RP6X(0, 0, 0, 0~1)	NU	RP0X(0, 0, 0, 0~1)	A(0, 0, 0, 0~22)	NU		NU	NU				EU_FMUL_ DLY		
cyc22	RP6X(0, 0, 0, 0~1)	NU	RP0X(0, 0, 0, 0~1)	A(0, 0, 0, 0~23)	NU	RP8X(0, 0, 0, 0~1)	NU	NU				EU_FADD[UB 1.5F-B]		
cyc23	RP6X(0, 0, 0, 0~1)	NU	RP0X(0, 0, 0, 0~1)	A(0, 0, 0, 0~24)	NU	NU	NU	NU				EU_FADD_ DLY		
cyc24	RP6X(0, 0, 0, 0~1)	RP8-(0, 0, 0, 0~1)	RP0X(0, 0, 0, 0~1)	A(0, 0, 0, 0~25)	NU	NU	NU	NU				EU_FMUL[UB A*B]		
cyc25	NU	NU	RP0X(0, 0, 0, 0~1)	A(0, 0, 0, 0~26)	NU		NU	NU				EU_FMUL_ DLY		
cyc26	RP9X(0, 0, 0, 0~1)	RP9X(0, 0, 0, 0~1)	RP0X(0, 0, 0, 0~1)	A(0, 0, 0, 0~27)	NU		NU	NU				EU_FMUL[UA UB A*B]		
cyc27	NU	NU	RP0X(0, 0, 0, 0~1)	A(0, 0, 0, 0~28)	NU		NU	NU				EU_END		
cyc28	R[10X(0, 0, 0, 0~1)	NU	RP0X(0, 0, 0, 0~1)	A(0, 0, 0, 0~29)	NU		NU	NU				EU_FMUL[UA A*D]		
cyc29	NU	NU	RP0X(0, 0, 0, 0~1)	NU	NU		NU	NU				EU_FMUL_ DLY		
cyc30	NU	NU	RP0X(0, 0, 0, 0~1)	NU	NU		NU	NU	R(0, 0, 0, 0~1)					EU_RESUL T

Fig. 23: ucode and data flow of floating-point division.